

1. Resolver el problema de clasificación de lesiones dermatológicas pero esta vez utilizando CNNs

Para ello, se replantea la implementación previamente desarrollada con un MLP, adaptándola a una arquitectura CNN lo más simple posible. El modelo está inspirado en AlexNet y se construye desde cero utilizando PyTorch, sin aplicar técnicas de transfer learning, data augmentation ni métodos avanzados de regularización.

La arquitectura empleada incluye:

- capas convolucionales
- funciones de activación ReLU
- operaciones de MaxPooling
- capas fully connected.

Además, las imágenes son redimensionadas a 128x128 píxeles con el objetivo de reducir el costo computacional del entrenamiento.

El modelo se entrena utilizando el optimizador Adam y la función de pérdida CrossEntropyLoss, adecuada para problemas de clasificación multiclase.

A partir de esta configuración básica, se obtienen los siguientes resultados, los cuales servirán como baseline inicial para futuras mejoras y comparaciones.

el archivo '01_baseline_cnn.ipynb' nos proporciona los siguientes resultados:

1		precision	recall	f1-score	support
2					
3	Actinic keratosis	0.41	0.65	0.50	20
4	Atopic Dermatitis	0.64	0.33	0.44	21
5	Benign keratosis	0.90	0.95	0.93	20
6	Dermatofibroma	0.47	0.40	0.43	20
7	Melanocytic nevus	0.62	0.75	0.68	20
8	Melanoma	0.47	0.45	0.46	20
9	Squamous cell carcinoma	0.25	0.25	0.25	20
10	Tinea Ringworm Candidiasis	0.55	0.55	0.55	20
11	Vascular lesion	0.94	0.80	0.86	20
12					
13	accuracy			0.57	181
14	macro avg	0.58	0.57	0.57	181
15	weighted avg	0.58	0.57	0.57	181

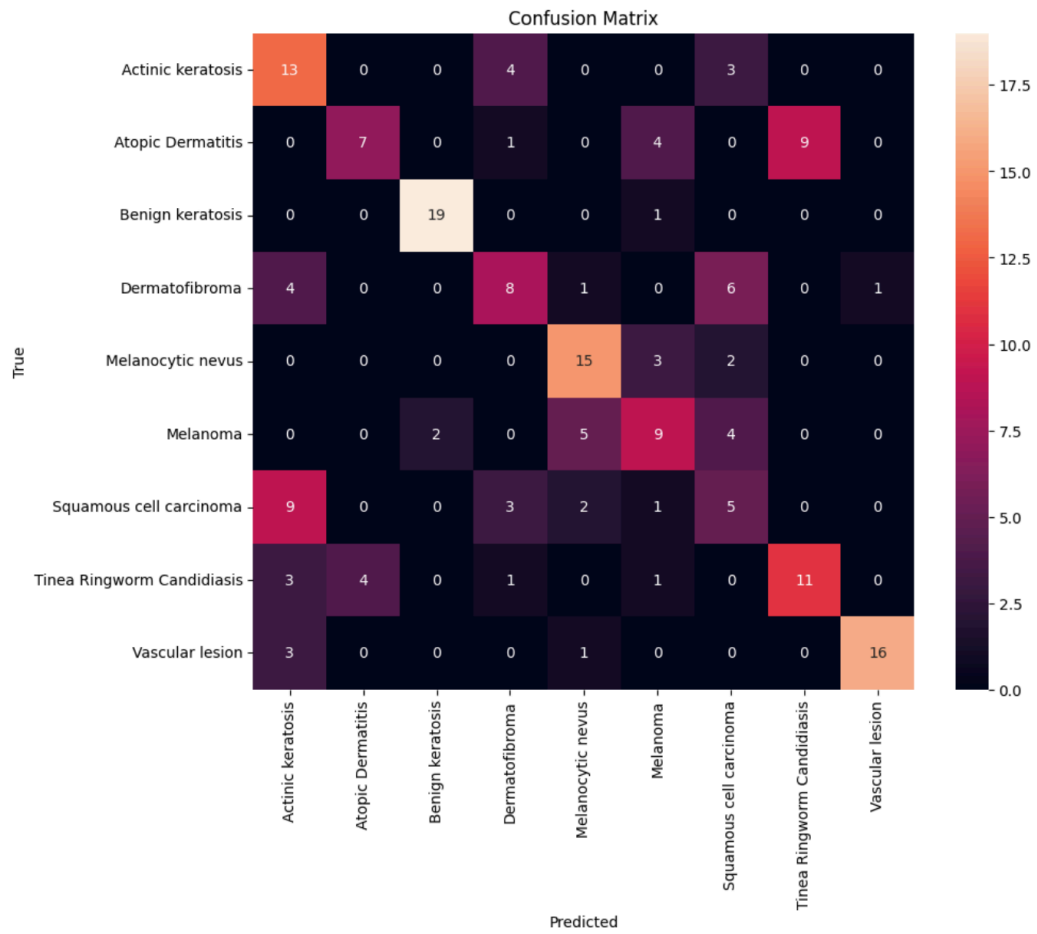


Figure 1: Matriz de Confusion

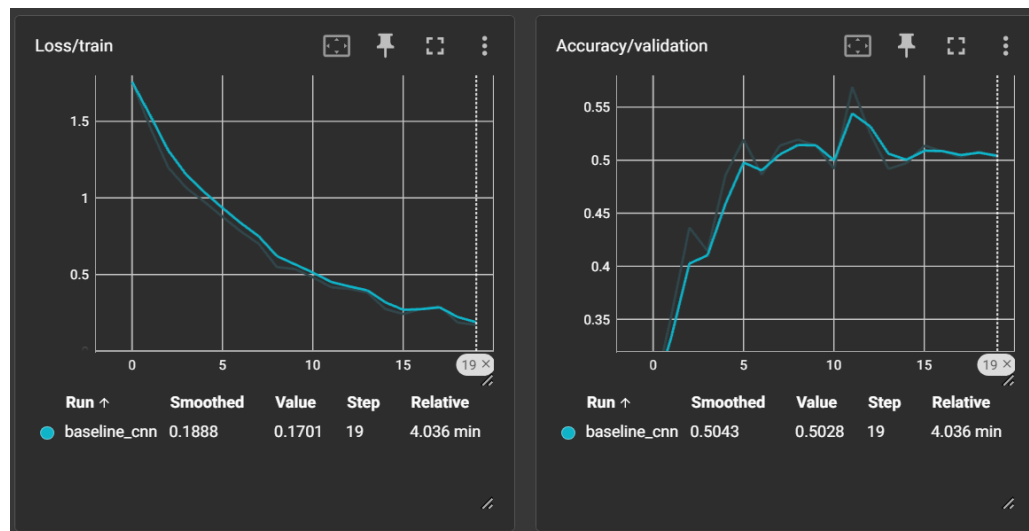


Figure 2: Iteraciones de baseline_cnn

2. Agregando augmentation

Modificando transform por:

```
1  train_transform = transforms.Compose([
2      transforms.Resize((128,128)),
3      transforms.RandomHorizontalFlip(p=0.5),
4      transforms.RandomRotation(20),
5      transforms.ColorJitter(
6          brightness=0.2
7      ),
8      transforms.ToTensor(),
9  ])
10
11 val_transform = transforms.Compose([
12     transforms.Resize((128,128)),
13     transforms.ToTensor(),
14 ])
```

Obtenemos los siguientes resultados

1		precision	recall	f1-score	support
2					
3	Actinic keratosis	0.50	0.65	0.57	20
4	Atopic Dermatitis	0.48	0.76	0.59	21
5	Benign keratosis	0.89	0.85	0.87	20
6	Dermatofibroma	0.61	0.55	0.58	20
7	Melanocytic nevus	0.67	0.90	0.77	20
8	Melanoma	0.43	0.45	0.44	20
9	Squamous cell carcinoma	0.27	0.15	0.19	20
10	Tinea Ringworm Candidiasis	0.38	0.15	0.21	20
11	Vascular lesion	0.94	0.85	0.89	20
12					
13	accuracy			0.59	181
14	macro avg	0.58	0.59	0.57	181
15	weighted avg	0.57	0.59	0.57	181

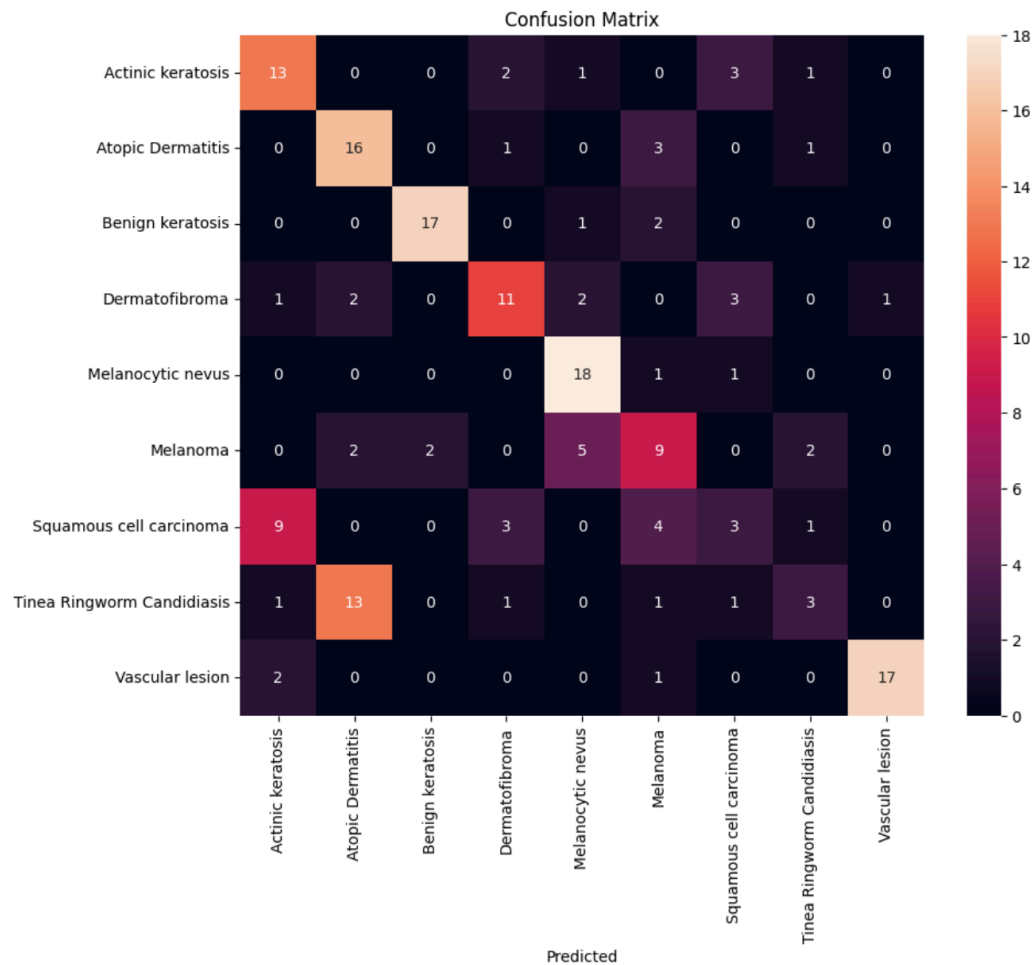


Figure 3: Matriz de Confusion

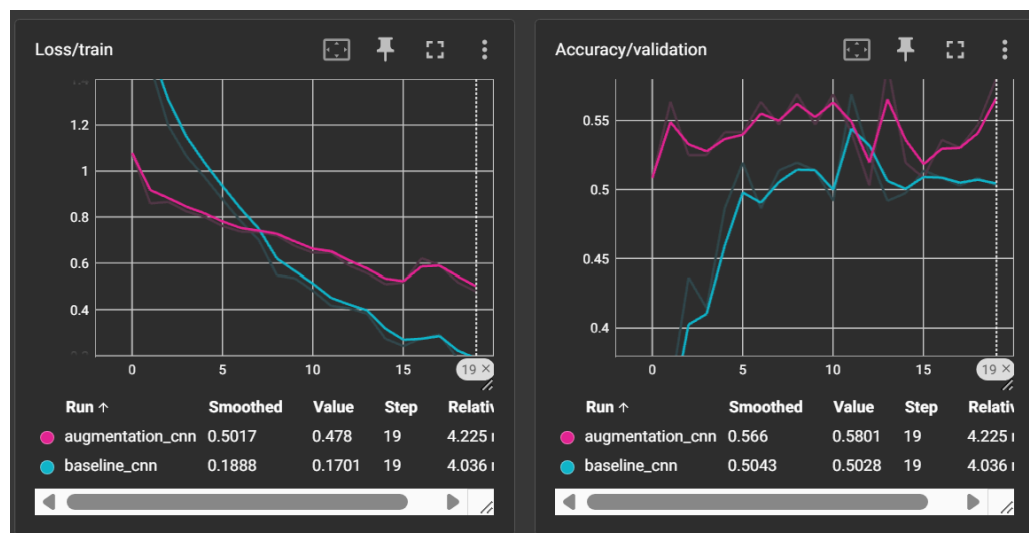


Figure 4: Iteraciones comparando base con augmented

3. Agregando regularizacion

Agergando normalizaciones y un dropout de 0.2 en el arhivo '03_regularization.ipynb', obtenemos los siguientes resultados:

1		precision	recall	f1-score	support
2					
3	Actinic keratosis	0.36	0.70	0.47	20
4	Atopic Dermatitis	0.53	0.81	0.64	21
5	Benign keratosis	0.95	0.95	0.95	20
6	Dermatofibroma	0.33	0.30	0.32	20
7	Melanocytic nevus	0.75	0.75	0.75	20
8	Melanoma	0.50	0.40	0.44	20
9	Squamous cell carcinoma	0.40	0.20	0.27	20
10	Tinea Ringworm Candidiasis	0.38	0.15	0.21	20
11	Vascular lesion	0.89	0.80	0.84	20
12					
13	accuracy			0.56	181
14	macro avg	0.57	0.56	0.54	181
15	weighted avg	0.57	0.56	0.54	181



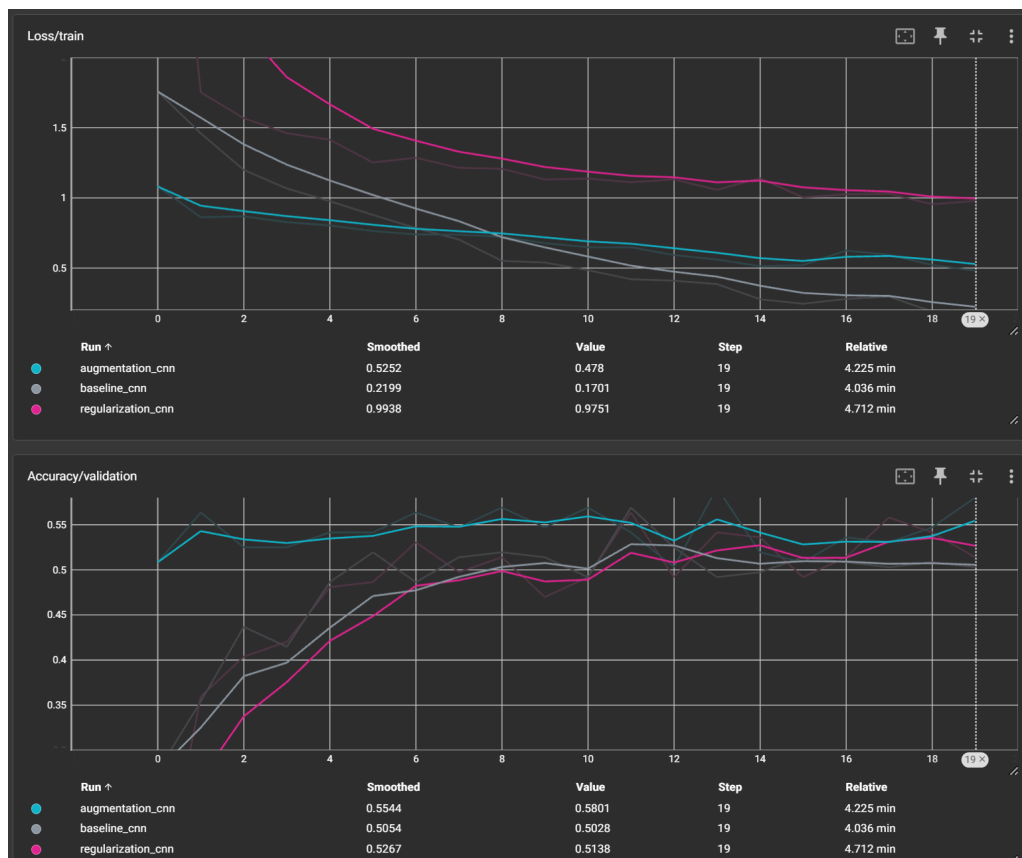


Figure 6: Iteraciones comparando regularizacion con

4. Búsqueda de Hiperparametros

Iterando los parametros con 10 iteraciones, obtenemos la siguiente tabla

1		lr	batch_size	dropout	optimizer	accuracy
2	24	0.0001	16	0.3	Adam	57.458564
3	30	0.0001	32	0.5	Adam	56.906077
4	28	0.0001	32	0.3	Adam	55.801105
5	17	0.0010	32	0.3	SGD	55.801105
6	23	0.0010	64	0.5	SGD	54.696133
7	13	0.0010	16	0.3	SGD	54.696133
8	34	0.0001	64	0.5	Adam	54.143646
9	21	0.0010	64	0.3	SGD	54.143646
10	32	0.0001	64	0.3	Adam	53.591160
11	12	0.0010	16	0.3	Adam	52.486188
12	19	0.0010	32	0.5	SGD	51.933702
13	26	0.0001	16	0.5	Adam	51.381215
14	16	0.0010	32	0.3	Adam	50.828729
15	15	0.0010	16	0.5	SGD	49.723757
16	22	0.0010	64	0.5	Adam	49.171271
17	25	0.0001	16	0.3	SGD	48.618785
18	29	0.0001	32	0.3	SGD	47.513812
19	9	0.0100	64	0.3	SGD	47.513812
20	27	0.0001	16	0.5	SGD	45.856354
21	5	0.0100	32	0.3	SGD	44.751381
22	14	0.0010	16	0.5	Adam	44.751381
23	31	0.0001	32	0.5	SGD	43.646409
24	18	0.0010	32	0.5	Adam	43.646409
25	11	0.0100	64	0.5	SGD	42.541436
26	20	0.0010	64	0.3	Adam	42.541436
27	33	0.0001	64	0.3	SGD	41.988950
28	35	0.0001	64	0.5	SGD	37.569061
29	7	0.0100	32	0.5	SGD	32.596685
30	1	0.0100	16	0.3	SGD	28.176796
31	3	0.0100	16	0.5	SGD	24.861878
32	8	0.0100	64	0.3	Adam	16.022099
33	4	0.0100	32	0.3	Adam	12.707182
34	10	0.0100	64	0.5	Adam	12.707182
35	0	0.0100	16	0.3	Adam	11.602210
36	2	0.0100	16	0.5	Adam	11.602210
37	6	0.0100	32	0.5	Adam	11.049724

Observamos que la mejor configuracion entonces, es con un ‘lr’: 0.0001, ‘batch_size’: 16, ‘dropout’: 0.3, ‘optimizer’: ‘Adam’ obteniendo un ‘accuracy’ de 57.4585635359116